Hochschule Konstanz
Fakultät Informatik

Software Architecture

Functional Style

Prof. Dr. Marko Boger

Scalachess

Main Abstractions of ScalaChess

Coordinate System

- Square (A1 .. H8)
- File(A..H)
- Rank(1..8)
- Side(QueenSide, KingSide)

Identity

- Color (Black/White)
- Role (Pawn, Rook, Knight..)
- PromotableRole
- Piece

Placement and Movement

- Board
- Bitboard
- History
- Status
- Position
- Move

BitBoard

ScalaChess makes use of 64 bit arithmetics.

This is fast, but hard to read.

Implement without bit operations!

```
object Board:
 val standard = Board(
   occupied = Bitboard(0xffff00000000ffffL),
   byColor = ByColor(
     white = Bitboard(0x000000000000ffffL),
     black = Bitboard(0xffff000000000000L)
   ),
   byRole = ByRole(
     pawn = Bitboard(0x00ff00000000ff00L),
     knight = Bitboard(0x4200000000000042L),
     bishop = Bitboard(0x2400000000000024L),
     rook = Bitboard(0x8100000000000081L),
     queen = Bitboard(0x0800000000000008L),
     king = Bitboard(0x1000000000000010L)
   )
 )
```

The Essence of Functional Style

- Immutability
  - Use val, not var
  - Deep-copy to change state
    - Case Class provides a copy with the same parameter as the constructor
    - Keep all state in constructor parameter
- Functions are objects
  - Functions as parameters
  - Functions can be assigned
  - Functions can be returned
- Expressions instead of Statements
- Recursion instead of loops

Closures

- A closed term is a function without free variables

```
def f1(x: Int) = x + 1              //> f1: (x: Int)Int
```

- An open term is a function with free variables

```
var c = 1                           //> c  : Int = 1
def f2(x: Int) = x + c              //> f2: (x: Int)Int
```

- A closure is the function value of the open term,
- thus a closure closes the open term at runtime, by capturing the bindings of its free variables

```
f2(5)                               //> res15: Int = 6
c = 10
f2(5)                               //> res16: Int = 15
```

Currying

- Turn a function with multiple arguments into a sequence of functions, each with a single argument.

```
def f3(c: Int) = (x: Int) => x + c  //> f3: (c: Int)Int => Int
 f3(1)(5)                           //> res17: Int = 6
 f3(10)(5)                          //> res18: Int = 15
 f3(10) { 5 }                       //> res19: Int = 15
 f3(10) {
   val z = 4
   f(z)
 }
```

Partially Applied Functions

- Applying a curried function to a part of its parameters
  - Returns a function with the missing parameters

```
def f4 = f3(10)          //> f4: => Int => Int

f4(5)                    //> res21: Int = 15
```

Putting it all together

- A Closure, a curried function

```
def f5(op:(Int, Int)=>Int) (x:Int, y:Int) = {
   op(x,y)
}       //> f5: (op: (Int, Int) => Int)(x: Int, y: Int)Int

def add(x:Int, y:Int) = x + y   //> add: (x: Int, y: Int)Int

f5(add) (5,1)                   //> res22: Int = 6
```

- A partially applied function

```
def f6 = f5(add) _              //> f6: => (Int, Int) => Int

f6(5,1)                         //> res23: Int = 6
```

A practical Example: msort

```
def msort[T](less: (T, T) => Boolean)(xs: List[T]): List[T] = {
  def merge(xs: List[T], ys: List[T]): List[T] =
    (xs, ys) match {
      case (Nil, _) => ys
      case (_, Nil) => xs
      case (x :: xs1, y :: ys1) =>
        if (less(x, y)) x :: merge(xs1, ys)
        else y :: merge(xs, ys1)
    }
  val n = xs.length / 2
  if (n == 0) xs
  else {
    val (ys, zs) = xs splitAt n
    merge(msort(less)(ys), msort(less)(zs))
  }
}        //> msort: [T](less: (T, T) => Boolean)(xs: List[T])List[T]
```

A practical Example: msort

```
def intsort = msort((x: Int, y: Int) => x < y) _
                    //> intsort: => List[Int] => List[Int]

intsort(List(9, 2, 5, 7, 3, 8))
                    //> res24: List[Int] = List(2, 3, 5, 7, 8, 9)

def reverseintsort = msort((x: Int, y: Int) => x > y) _
                    //> reverseintsort: => List[Int] => List[Int]
reverseintsort(List(9, 2, 5, 7, 3, 8))
                    //> res25: List[Int] = List(9, 8, 7, 5, 3, 2)

def stringsort = msort((s1: String, s2: String) => s1.length <
s2.length) _
                    //> stringsort: => List[String] => List[String]
stringsort(List("coffee", "tee", "beer", "orangejuice"))
                    //> res26: List[String] = List(tee, beer, coffee, orangejuice)
```

An even more practical Example

```
def sum(list:List[Int]):Int = list match {
  case Nil => 0
  case _ => list.head + sum(list.tail)
}                          //> sum: (list: List[Int])Int


sum(List(1,2,3,4,5))       //> res27: Int = 15
```

An even more practical Example

```
def product(list:List[Int]):Int = list match {
  case Nil => 1
  case _ => list.head * product(list.tail)
}
                    //> product: (list: List[Int])Int


product(List(1,2,3,4,5))
                    //> res28: Int = 120
```

An even more practical Example

```
def sum2(list:List[Int])=list.foldLeft(0)(_+_)
                                    //> sum2: (list: List[Int])Int
sum2(List(1,2,3,4,5))          //> res29: Int = 15

List(1,2,3,4,5).foldLeft(0)(_+_) //> res30: Int = 15

def product2(list:List[Int])=list.foldLeft(1)(_*_)
                                    //> product2: (list: List[Int])Int

product2(List(1,2,3,4,5))  //> res31: Int = 120

List(1,2,3,4,5).foldLeft(1)(_*_) //> res32: Int = 120
```

Conclusion

- Closures are essentially a way to pre-configure functions
  - Functions like sort, fold or reduce make use of it
- Currying allows to separate parameters
- Partially Applied Functions allow to apply one parameter separately
- Together they empower Closures

Task 1

Make use of functional style

- Short classes. No Getter and Setter. Constructor as part of declaration
- No Null and null. Instead use Option
- Use standard collections
- No var in model layer. Use val
- Functions as parameter
- Function chaining
- No state
- Recursion
- Closure, Currying, Partially Applied Functions

Example for Currying

Without Currying

```
def checkMillBlack(cell1: Cell, cell2: Cell, cell3: Cell): Boolean = {
  (cell1.getContent.whichColor == Color.black && cell2.getContent.whichColor == Color.black
    && cell3.getContent.whichColor == Color.black)
}

def checkMillWhite(cell1: Cell, cell2: Cell, cell3: Cell): Boolean = {
  (cell1.getContent.whichColor == Color.white && cell2.getContent.whichColor == Color.white
    && cell3.getContent.whichColor == Color.white)
}
```

With Currying

```
def checkMill(color:Color)(cell1: Cell, cell2: Cell, cell3: Cell): Boolean = {
  (cell1.getContent.whichColor == color && cell2.getContent.whichColor == color
    && cell3.getContent.whichColor == color)
}
def checkMillWhite(cell1: Cell, cell2: Cell, cell3: Cell): Boolean = checkMill(Color.White) _
def checkMillBlack(cell1: Cell, cell2: Cell, cell3: Cell): Boolean = checkMill(Color.Black) _
```

Without Closure and Recursion

```
def checkRow(row: Int, color: Color): Boolean = {
  var pieces = new ListBuffer[Color]()
  for (col <- 0 until sizeOfCols) {
    pieces += cell(row, col).color
  }
  var counter = 0
  breakable {
    for (elem <- pieces) {
      if (elem.equals(color)) counter += 1 else counter
= 0
      if(counter >= 4) break
    }
  }



  if (counter >= 4) true else false
}
```

```
def checkCols(col: Int, color: Color): Boolean = {
  var pieces = new ListBuffer[Color]()
  for (row <- 0 until sizeOfRows) {
    pieces += cell(row, col).color
  }
  var counter = 0
  breakable {
    for (elem <- pieces) {
      if (elem.equals(color)) counter += 1 else counter
= 0
      if(counter >= 4) break
    }
  }



  if (counter >= 4) true else false


}
```

With Closures and Recursion

```
 def hasWon(directionFunc : (Int, Int) => (Int, Int), color :
Color, counter : Int, currentIdx : (Int, Int)): Boolean ={
   if(counter == 4) return true
   val newIndicies = directionFunc(currentIdx._1, currentIdx._2)
   if(newIndicies._1 >= sizeOfRows || newIndicies._2 >=
sizeOfCols) return false
   var newCounter = 0;
   if(cell(newIndicies._1, newIndicies._2).color == color){
     newCounter = counter +1;
   }
   hasWon(directionFunc, color, newCounter, newIndicies)
 }
```

```
def checkRow(row: Int, color: Color): Boolean = {
  hasWon((y,x) => (y, x+1), color, 0, (row,-1))


def checkCols(col: Int, color: Color): Boolean = {
  hasWon((y,x) => (y+1, x), color, 0, (-1,col))
```

Production Code

In production code, often the bad cases need to be considered as well, like wrong input, network failure etc.

This often requires a lot of if-cases.

An if statement is somewhat like a rail switch.

An Example using Bottles

```
class Bottle {
 var empty=false
def consume = {
 println(" consuming... ")
 empty = true
 this
}
override def toString= if (empty) "b" else "B"
}

class Pack(val bottles:List[Bottle]) {
 def map(f:Bottle => Bottle) = bottles.map(bottle => f(bottle))
 override def toString="UUUU"
}

class Crate(val packs:List[Pack]) {
 def map(f:Pack => Pack) = packs.map(pack => f(pack))
 def flatMap(f:Pack => List[Bottle]) = packs.flatMap(pack => f(pack))
 override def toString="L_____J"
}
```

The Good Case - Java Style

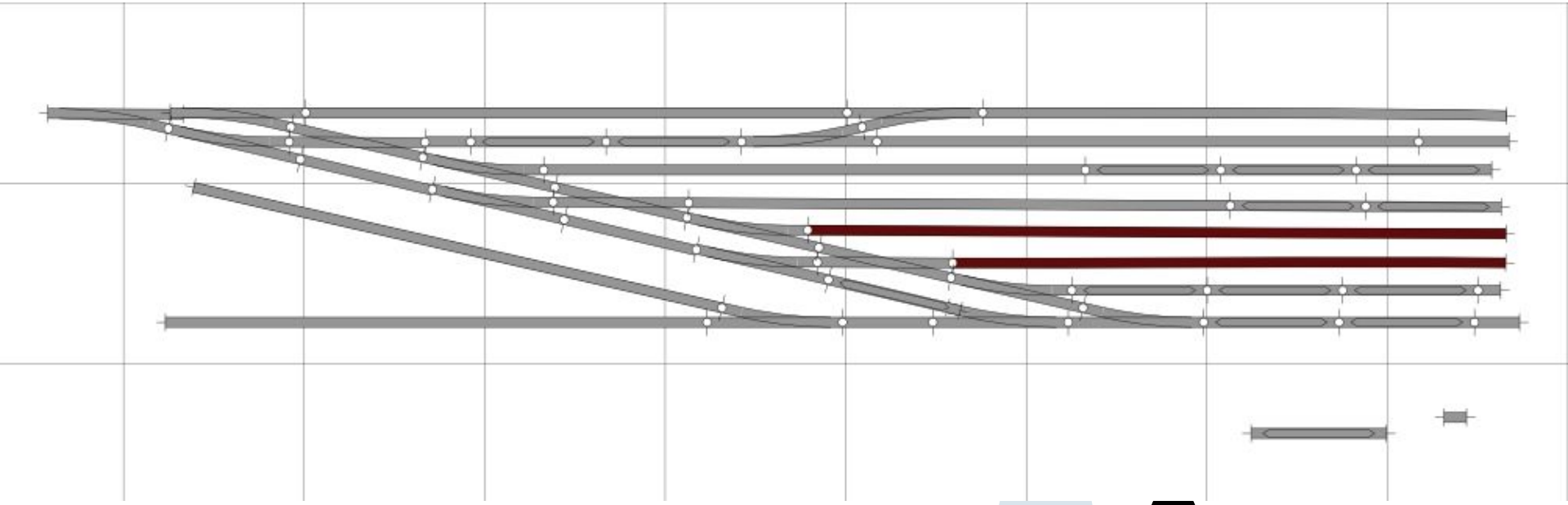
```
val pack1 = new Pack(List(new Bottle, new Bottle, new Bottle, new Bottle))

val pack2 = new Pack(List(new Bottle, new Bottle, new Bottle, new Bottle))

val crate1 = new Crate(List(pack1, pack2))

def consumeAll(crate:Crate) = {
 var packs = crate.packs
 packs.foreach {pack =>
  var bottles = pack.bottles
  bottles.foreach { bottle =>
   bottle.consume
  }
 }
}

consumeAll(crate1)
```

The Bad Case - Java Style

```
val pack3 = new Pack(List(new Bottle, null, new Bottle, new Bottle))
val pack4 = new Pack(List(new Bottle, new Bottle, null, new Bottle))
val crate2 = new Crate(List(pack3, pack4))
def consumeAllAssumeNull(crate:Crate) = {
 if (crate != null) {
  var packs = crate2.packs
  if (packs != null) {
    packs.foreach { pack =>
     if (pack != null) {
      var bottles = pack.bottles
      if (bottles != null) {
       bottles.foreach { bottle =>
        if (bottle != null) bottle.consume
        else println ("Found a null for bottle")
       }
      } else println ("Found a null for bottles")
     } else println ("Found a null for pack")
    }
  } else println ("Found a null for packs")
 }  else println ("Found a null for crate")
}
consumeAllAssumeNull(crate2)
```

Rail Metaphor

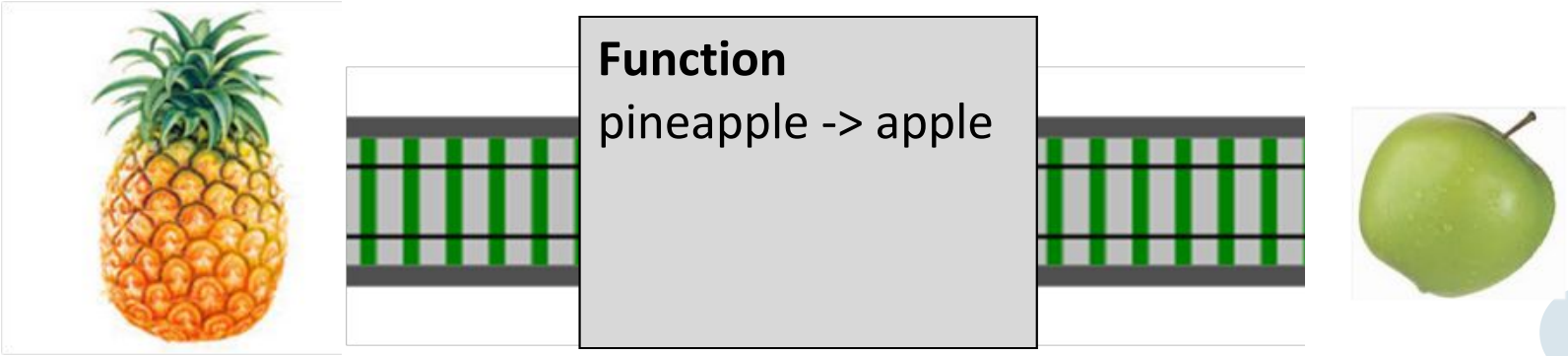
Every if branches off the code, in a similar way as rail switches branches off a rail track. This can lead to very high complexity.



Rails as Metaphor for Functional Programming

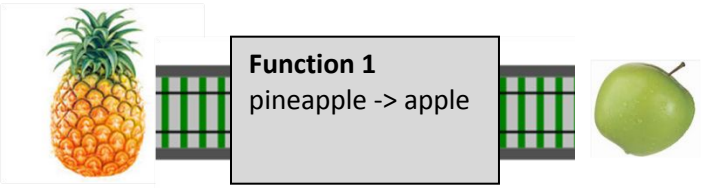
A function can be compared to a track and a transformative tunel.

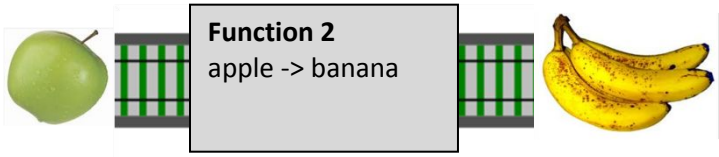
One thing goes in as input, transformation takes place, and out comes a new thing. Input and output have a type.



Composition

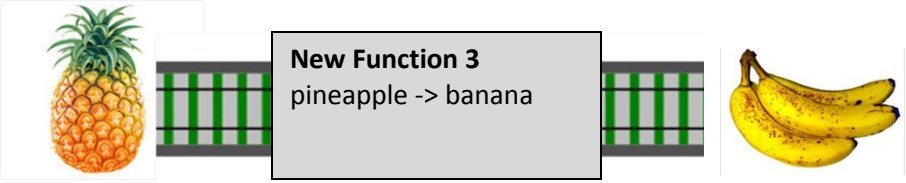
If type of input of one function is compatible with the input of another function, these can be composed.





Composite

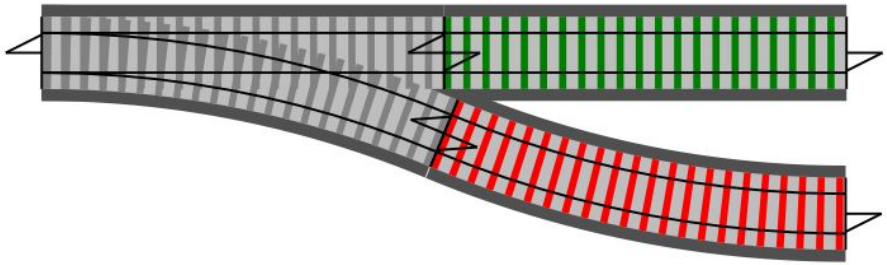
The composed thing is a new function.



Two output Types

Some functions have two different types of output. Like Option:

Input->

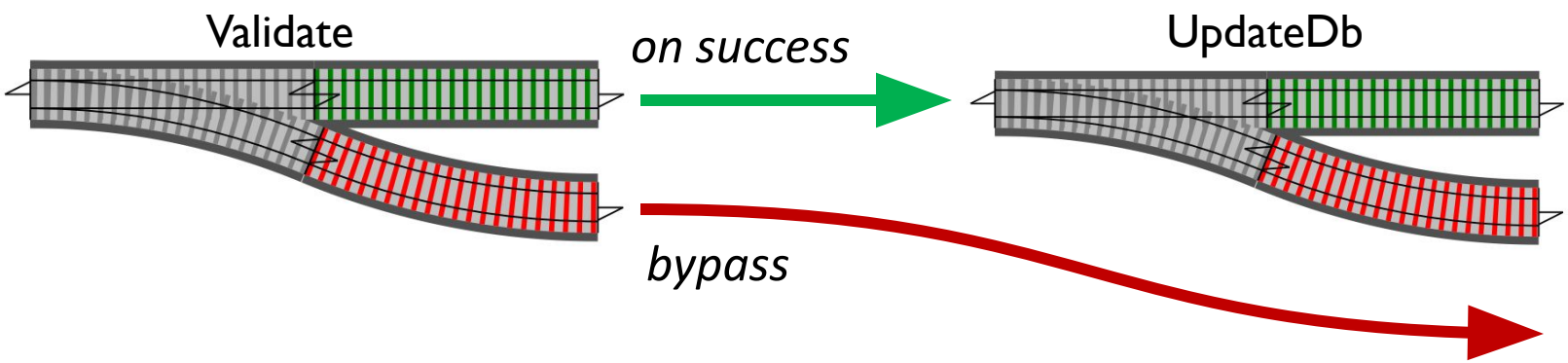


Success

Failure

Connecting Switches

Composing these may seem difficult at first. And if not done right, this can lead to a lot of tracks and high complexity.

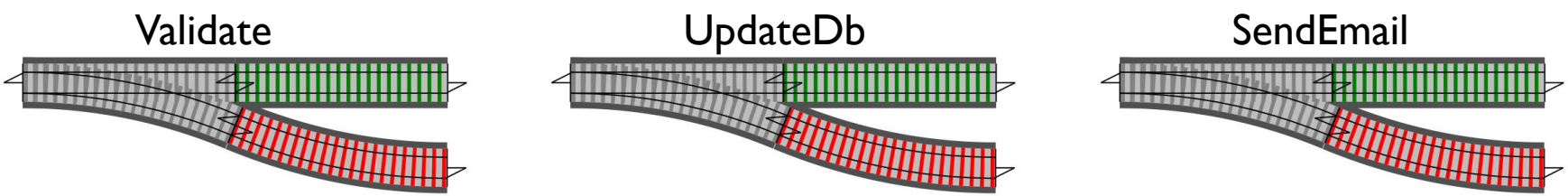


Connecting Switches right

But if done in a clever way, these can be connected very elegantly.

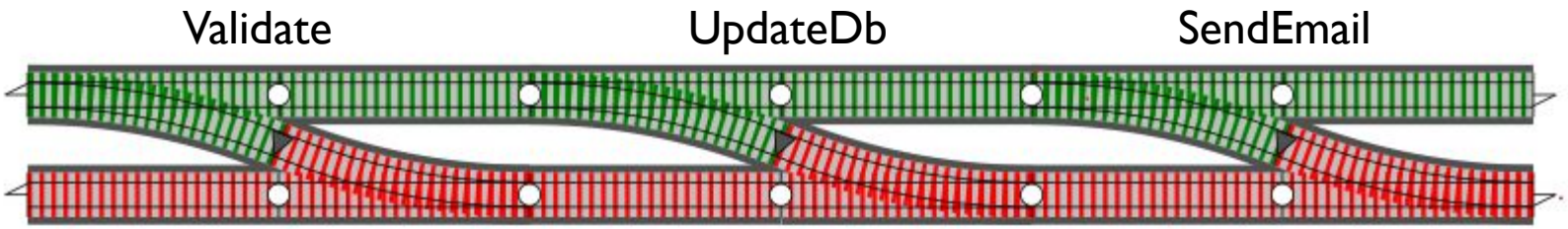
Connecting many Switches

Can this approach scale to a larger number of switches?



Connecting many Switches right

We can establish a rail track for the good case and one for the bad case. Once a train is on the bad track, it can never go back to the good track, but from the good track it can move to the bad track at various points.



Monad - A Functional Design Pattern

A Monad is a functional design pattern.

Basically it is a container for a data structure with an algebraic structure.

A simple such algebraic structure is a Monoid.

Groups and Abelian Groups are also Monoids.

Algebraic Structures

Monoid

A monoid is an algebraic structure with a single associative binary operation and an identity element. Monoids are semigroups with identity.

Group-like structures. The entries say whether the property is *required*.

	Totality*	Associativity	Identity	Divisibility	Commutativity
Semicategory	No	Yes	No	No	No
Category	No	Yes	Yes	No	No
Groupoid	No	Yes	Yes	Yes	No
Magma	Yes	No	No	No	No
Quasigroup	Yes	No	No	Yes	No
Loop	Yes	No	Yes	Yes	No
Semigroup	Yes	Yes	No	No	No
Monoid	Yes	Yes	Yes	No	No
Group	Yes	Yes	Yes	Yes	No
Abelian Group	Yes	Yes	Yes	Yes	Yes

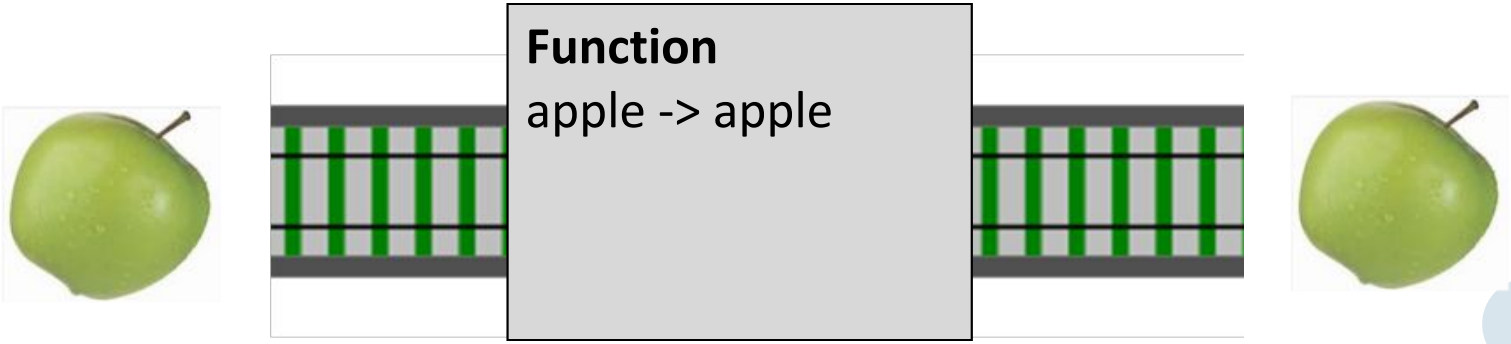
*Closure, which is used in many sources, is an equivalent axiom to totality, though defined differently.

Rules for Monoids

- Associativity
  - For all a, b and c in S, the equation (a • b) • c = a • (b • c) holds.
- Identity element
  - There exists an element e in S such that for every element a in S, the equations e • a = a • e = a hold.
- Examples
  - Strings with concat and empty word
  - 12 Hour Clock with add and zero
  - Integers with add and zero
  - Integers with modulo and 1
  - Lists with concat and ()
  - Sets

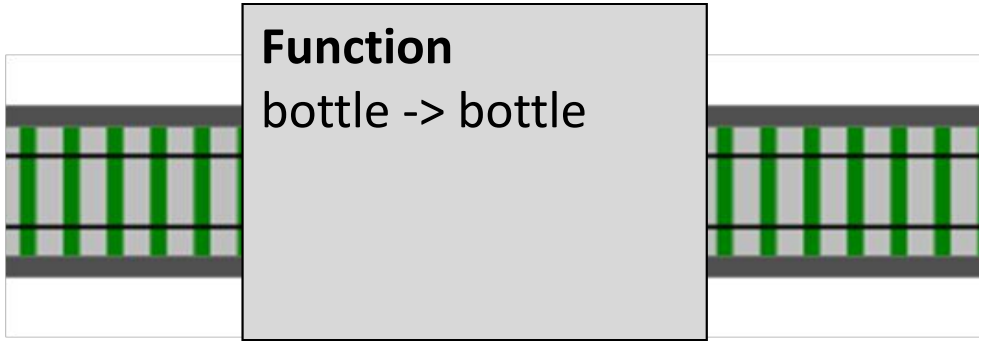
Monoids in terms of Rails

A Monoid is a function from a Set into the same Set



Monoids in terms of Rails

A Monoid is a function from a Set into the same Set



Monads and Monoids

The Monad acts as a container for a Monoid.

We can take an element out of the container, apply a function or transformation to the element, and put it into a new container.

For example, we can take a bottle from a crate, put a label on it, and put it into a new crate.

Option and Try are Monads

- Try and Option are Monads for handling a single content
- Think of it as a container with just one content for secure transportation
  - Like a box for a bottle of wine

Option is a Monad

Option can contain Some(thing) or None

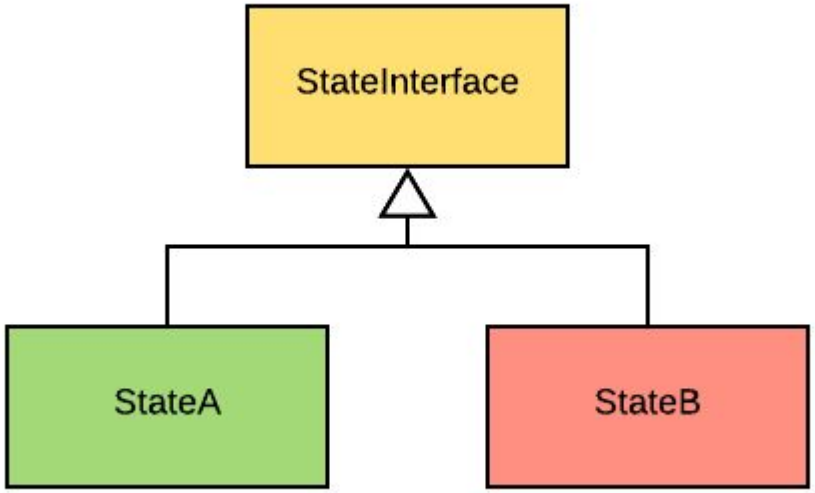
Try is a Monad

- A Try can contain a Success or a Failure

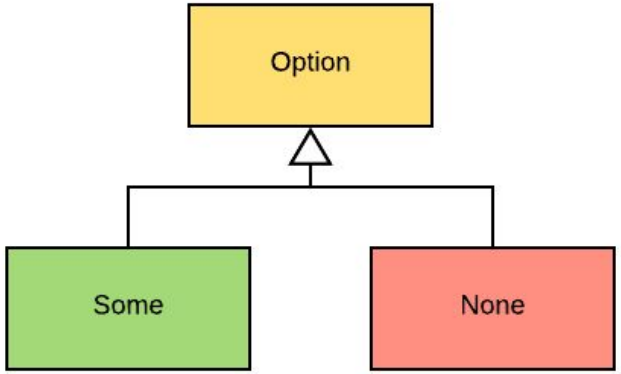
The State Pattern applied in Monads

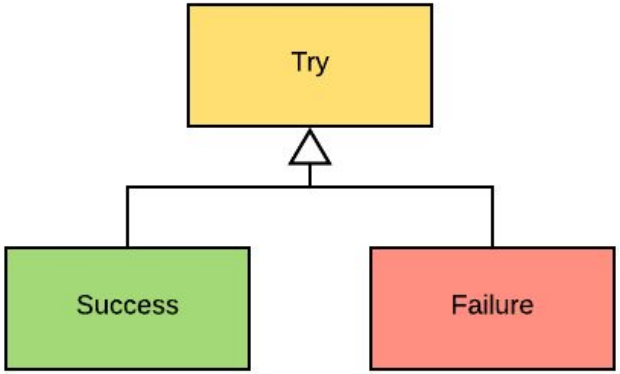
Take a look again at the structure of the State Pattern.

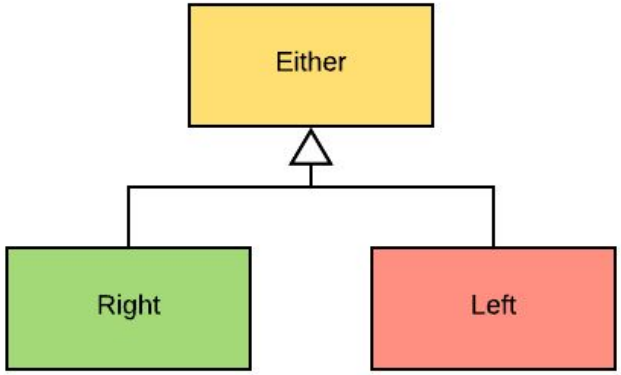
Monads use this structure and typically distinguish a “good” and a “bad” state.

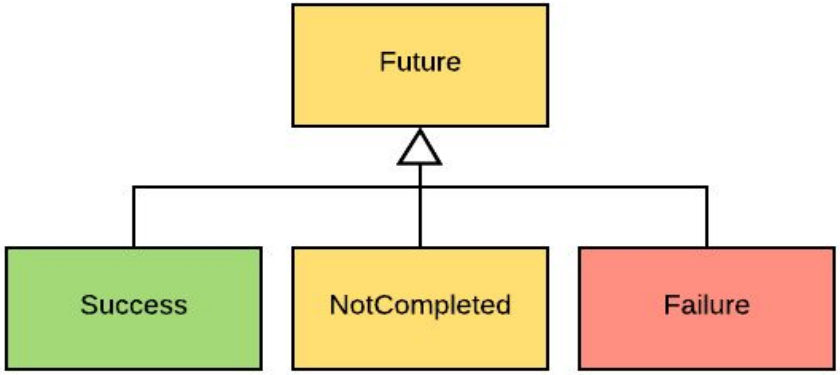


Option, Try, Future and Either





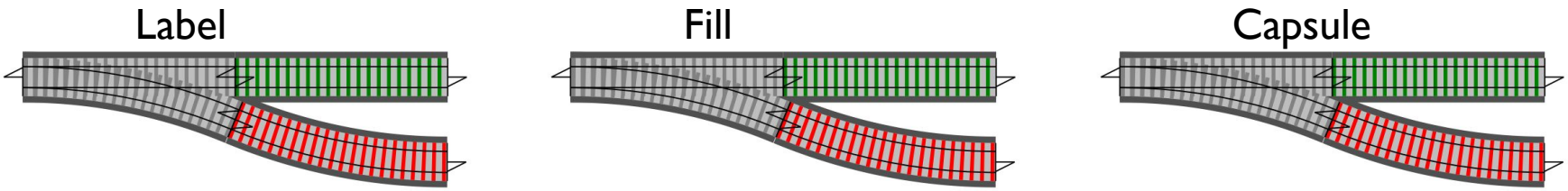




Connecting Monads

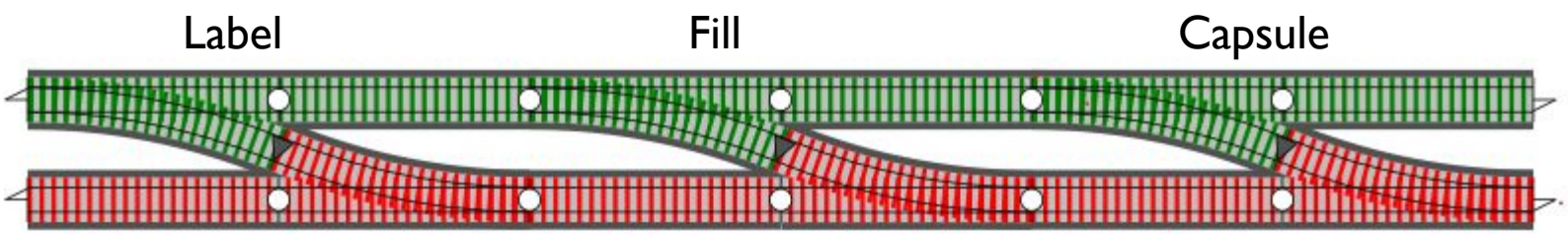
The Monoid inside a Monad converts from a Set to the same Set, eg. from Bottle to Bottle.

The Monad differentiates the good case from the bad case and makes it easy to handle.



Connecting many Switches right

We can establish a rail track for the good case and one for the bad case. Once a train is on the bad track, it can never go back to the good track, but from the good track it can move to the bad track at various points.



One-Track Code

```
object BottleState extends Enumeration {
type BottleState = Value
val Empty, Labeled, Filled, Capsuled, Consumed = Value
}
import BottleState._

case class Bottle(state: BottleState=Empty) {
def label: Bottle = copy(state= Labeled)
def fill:Bottle = copy(state= Filled)
def capsule:Bottle = copy(state= Capsuled)
def consume:Bottle = {
 println(" consuming... ")
 copy(Consumed)
}
}

new Bottle().label.fill.capsule
```

Two-Track Code

```
case class MaybeBottle(bottle:Option[Bottle]) {
def label : MaybeBottle = bottle  match {
 case Some(bottle:Bottle) => copy( Some(bottle.label))
 case None => copy(None)
}
def fill: MaybeBottle = bottle  match {
 case Some(bottle:Bottle) => copy( Some(bottle.fill))
 case None => copy(None)
}
def capsule: MaybeBottle = bottle  match {
 case Some(bottle:Bottle) => copy( Some(bottle.capsule))
 case None => copy(None)
}
}

MaybeBottle(Some(new Bottle)).label.fill.capsule
MaybeBottle(None).label.fill.capsule
```

For-Comprehension for unpacking Monads

If the Monad is a container for anything, the for-comprehension is used to unpack that from the container, process it in some way and package it again in a new Monad.

For-Comprehension is syntactic Sugar for map

A simple for-expression with one Generator is expanded to a map function

```
// Translation of For(1)
for (x <- e1) yield toValue(x)

// translated from compiler to:
e1.map(x => toValue(x) )
```

For can be used for any data structure with map

Complex for is translated to map, flatmap and filter

```
for {
  i <- 1 to n
  j <- 1 to i
  if isEven(i + j)
} yield(i, j)
```

is translated to

```
(1 to n) flatMap ( i =>
  (1 to i) filter (j => isEven(i+j))
    map (j => (i, j)
  )
)
```

Bottle using Monads

```
class PackT[T](val bottles:List[T]) {
 def map(f:T => T) = bottles.map(bottle => f(bottle))
 override def toString="UUUU"
}

class CrateT[T](val packs:List[T]) {
 def map(f:T => T) = packs.map(pack => f(pack))
 def flatMap(f:T => List[Bottle]) = packs.flatMap(pack => f(pack))
 override def toString="L_____J"
}
// Option already exists. This is a sketch of an implementation.
trait Option[Bottle] {
def map(f:Bottle => Bottle):Option[Bottle]
}
case class Some[Bottle](val b:Bottle) extends Option[Bottle] {
def map(f:Bottle => Bottle) = new Some(f(b))
}
case class None[Bottle]() extends Option[Bottle] {
def map(f:Bottle => Bottle) = new None
}
```

Using For to unpack the Monad

```
val pack5= new PackT( List(Some(new Bottle), None, Some(new Bottle), Some(new Bottle)))
val pack6= new PackT( List(Some(new Bottle), Some(new Bottle), None, Some(new Bottle)))
val crate3 = new CrateT(List(Some(pack5),Some(pack6)))


def consumeAllAssumeNoneWithFor(pack:PackT[Option[Bottle]]) = {
 for (
    bottle <- pack.bottles) yield bottle match {
  case Some(b) => b.consume
  case None => println("Found None")
 }
}
consumeAllAssumeNoneWithFor(pack6)
```

Same Code without Monads

```
def consumeAllAssumeNull(crate:Crate) = {
 if (crate != null) {
  var packs = crate2.packs
  if (packs != null) {
    packs.foreach { pack =>
     if (pack != null) {
      var bottles = pack.bottles
      if (bottles != null) {
       bottles.foreach { bottle =>
        if (bottle != null) bottle.consume
        else println ("Found a null for bottle")
       }
      } else println ("Found a null for bottles")
     } else println ("Found a null for pack")
    }
  } else println ("Found a null for packs")
 }  else println ("Found a null for crate")
}
```

Task 2

Avoid the use of null and Null. Use Option instead.

Also avoid using try-catch, use the Try-Monad instead.

Use the For-Comprehension to un-pack monads.

Try to apply the two-track pattern to your code.